Basic (critique your own Stage-1 design) — Hints

Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order against your own written design before reading ahead.

- Hint 1 — The idea, and the exact check
- Hint 2 — The plan, and a worked example

Hint 1 — The idea, and the exact check

Doc16 frames this directly: "requirements → architecture is matching, not inspiration." The self-review technique is to build a **traceability table** — walk your design's component list and, for each one, force yourself to name the *specific* requirement (not a general good practice) it satisfies. Two failure patterns to watch for while doing this:

- **Overbuilt complexity** — a piece justified by "this is what real production systems have" rather than by anything in your actual requirements. A caching layer for traffic you don't have yet is the canonical example.
- **Solving a problem you don't have** — a piece defends against a failure mode your requirements never mentioned as a real risk.

Then run the check in the reverse direction too: for each *non-functional* requirement (speed, cost, reliability target), confirm at least one component actually addresses it. A requirement with nothing satisfying it is a gap, not just noise — this catches under-building, which is just as common as over-building and easier to miss because nothing "looks wrong" about a design that's simply incomplete.

Go through your own component list, both directions, before Hint 2.

The bias pass — is the requirement column honest?

Here's the trap in doing this review on your own design: you're the same person who chose every component, so you're also the most motivated person alive to find a requirement that justifies keeping it. A traceability table filled out by the person who built the thing it's tracing isn't automatically honest — it can be gamed without you noticing, by quietly writing the "requirement" column to match the component you already wanted, instead of checking the component against a requirement that existed independently first.

The real self-review question isn't just "can I name a requirement for this" — it's **"did I write this requirement down before or after I built the component it's justifying?"** If you can't remember

requiring redaction before you designed a redaction layer, and you're now retroactively pointing at "must never expose raw customer data" to justify it, that's not a failure of the check — that's the check working, catching a rationalization instead of a match. The honest fix isn't to delete the requirement (it's probably real) — it's to be suspicious of any row you filled in *fast* and *confidently*, since post-hoc justifications come easily precisely because you're good at constructing them.

A concrete way to catch this: for your 2-3 most confident rows, try to state the requirement in a form you'd have written *before* seeing your own design — if you can't, or the wording suspiciously echoes your component's own name, treat that row as unverified until you check it against the actual written requirements document, not your memory of it.

Hint 2 — The plan, and a worked example

```
build a traceability table: Component | Requirement it satisfies

for each component:
    write the specific requirement (not a general principle)
    if empty/vague: classify as overbuilt complexity, or a fix for a
    non-existent risk
    decide: cut, defer with a written note, or find the real requirement
    you missed

check the reverse direction too:
    for each NON-FUNCTIONAL requirement (speed, cost, reliability target),
    confirm at least one component in your design actually addresses it
    — a requirement with nothing satisfying it is a gap
```

Worked example row:

```
Component: a caching layer in front of the retriever
Stated requirement: "must handle 50,000 requests/day, p95 latency
    under 2 seconds"
Verdict: KEEP -- the latency target is real and stated, and a cache
    directly serves it.

Component: a second, redundant fact-checker agent running in parallel
    with the first
Stated requirement: none found
Verdict: CUT (or defer with a note) -- "more checking seems safer" is
    not the same as a requirement calling for it.
```

The bias pass, as a plan:

The same worked example, with the bias check added on top:

```
Component: field-level redaction before any data reaches the LLM
Stated requirement (as I'd write it now): "must never expose raw
    customer data to the LLM"

Bias check: did I write this requirement down BEFORE designing the
    redaction layer, or am I pointing at it now because it happens to
    match what I already built?
    -> if this was in the live requirements handoff, it's real: KEEP,
       confidently.
    -> if I can't recall it being stated, and I'm inferring it from
       "well, customer data is sensitive, so obviously..." -- that's
       MY reasoning, not a requirement. Flag it: still probably the
       right call, but check it against the actual requirements
       document before treating it as settled.

Component: an admin dashboard with 12 chart types
Stated requirement: "watching/monitoring" (generic, from the build list)
Bias check: this one is easy to catch -- "watching/monitoring" doesn't
    specify 12 chart types, or any number. The gap between the vague
    ask and the specific thing built is the tell.
Verdict: CUT down to 3-4 charts tied to actual non-functional targets.
```

Run the bias check against your own most confident row first — that's usually where a rationalization hides, precisely because it felt too obvious to question. Then check the Solution.

Full solution: Show me the solution